

NL-to-SQL Backend Engine

Phase 4 Architecture Walkthrough & Technical Jury Defense Guide

System Overview: This document provides an in-depth, code-level architectural walkthrough of the nl2sql-backend service. It breaks down each core file in order of runtime execution, covering engineering rationale, function-level mechanics, limitations, and model jury Q&A.; Special focus is given to **deterministic AST validation**, **prompt engineering mechanics**, and **crash-proof pipeline resilience**.

1. app/config.py — Configuration & Environment Bootstrap

What It Does: Centralizes file-system path resolution and environment variable loading across the backend. It guarantees deterministic absolute paths for databases and data folders while enforcing a strict fail-fast validation check on the Gemini API key upon startup.

Key Mechanics & Objects:

- **Absolute Path Anchoring:** Uses `Path(__file__).resolve().parent.parent` to pin `BASE_DIR`, `DATA_DIR`, and `META_DB_PATH`, ensuring scripts work regardless of current working directory.
- **Hierarchical .env Discovery:** Checks `nl2sql-backend/.env` first, then falls back to workspace root `.env` before executing `load_dotenv()`.
- **Fail-Fast Validation:** Reads `GEMINI_API_KEY = os.getenv('GEMINI_API_KEY')`. If empty or absent, immediately halts execution with a `ValueError`.
- **Database URI Definitions:** Preconfigures connection strings for `DATABASE_URL` (SQLite meta-database) and file pointers to demo databases (`demo_hospital.db`, `demo_ecommerce.db`).

Design Decision: Built on the *Fail-Fast Principle*. A system should refuse to boot if required secrets or directories are missing, rather than failing silently mid-demo when a user triggers their first query.

Honest Limitation: Configuration is loaded statically into process memory once at startup. Changing environment variables requires a complete process restart; there is no live reload or dynamic vault integration.

Judge Question: "Why hardcode SQLite connection URLs and file paths in config.py instead of passing them dynamically per tenant?"

Model Answer: "For this prototype, our priority was zero-friction evaluation with self-contained demo databases (hospital and ecommerce). In an enterprise multi-tenant release, config.py would only declare engine pool defaults, while tenant connection strings would be encrypted and dynamically resolved via tenant headers."

2. app/models/meta_db.py — System Persistence & Audit Schema

What It Does: Defines the relational database schema and SQLAlchemy ORM models for tracking system metadata, completely decoupled from business data. It maintains logs of active sessions, conversation threads, generated SQL history, and execution audit trails inside `data/meta.db`.

ORM Models & Functions:

- **SessionModel (sessions):** Tracks connection sessions (id UUID string PK, db_type, connected_at timestamp).
- **ConversationModel (conversations):** Establishes thread groupings (id int PK, session_id FK, created_at).

- **QueryHistoryModel (query_history):** Persists every query translation (id, session_id FK, nl_query Text, generated_sql Text, confidence Float, query_type, created_at).
- **AuditLogModel (audit_log):** Captures actual executed SQL, affected row counts, and timestamps for governance.
- **init_db():** Calls Base.metadata.create_all(bind=engine) to ensure tables exist prior to handling traffic.
- **get_db_session():** FastAPI dependency generator that yields a SQLAlchemy SessionLocal() instance and ensures deterministic closing in a finally block.

Design Decision: Complete isolation between metadata and business data. Recording LLM translation history in an independent metadata store ensures we never pollute the user's business database while retaining full telemetry for fine-tuning.

Honest Limitation: SQLite file-level locking. Under high concurrent write volumes from multiple simultaneous users, SQLite write contention can produce database is locked timeouts. Production should use PostgreSQL.

Judge Question: "Why did you set connect_args={'check_same_thread': False} in your SQLite engine configuration?"

Model Answer: "FastAPI processes requests across an asynchronous thread pool. Python's default SQLite driver raises a ProgrammingError if a connection created in one thread is accessed in another. Setting check_same_thread=False, paired with scoped session dependency injection (get_db_session()), enables safe multithreaded operation."

3. app/services/session_store.py — In-Memory Session & Schema Cache

What It Does: Provides an ultra-fast in-memory cache for active database connection metadata. It keeps target database paths, table listings, and extracted column schemas readily available in RAM so query translation does not re-inspect the database on every prompt.

Key Functions:

- **SESSION_STORE:** Global module dictionary (Dict[str, Dict[str, Any]]) storing session state.
- **get_session(session_id):** Receives session UUID string; returns session metadata dictionary (or None).
- **set_session(session_id, data):** Receives session UUID and dictionary containing schema, db path, tables; writes directly to memory.
- **remove_session(session_id):** Evicts session data safely using SESSION_STORE.pop(session_id, None).

Design Decision: Database schema introspection (querying PRAGMA table info across dozens of tables) takes 20-50ms over disk I/O. Caching the structured schema in RAM reduces prompt assembly time to sub-millisecond speeds.

Honest Limitation: State is volatile and tied to a single process. Restarting the server wipes active sessions, and running multiple Uvicorn worker processes creates split-brain state where sessions are not shared.

Judge Question: "What happens to session_store if we spin up 4 Uvicorn worker processes or deploy behind a load balancer?"

Model Answer: "In a multi-process or multi-pod setup, an in-memory dictionary cannot be shared across processes. To scale horizontally, we would replace the internal dictionary with a Redis-backed session store using the exact same function signatures (get_session, set_session), applying an automatic 1-hour TTL."

4. app/routers/connect_db.py — Database Onboarding & Schema Discovery

What It Does: Exposes the POST /api/connect-db endpoint. It verifies the target database exists, uses SQLAlchemy's schema inspection engine to discover all tables and column types, logs the session in meta.db, and primes the in-memory cache.

Execution Flow (connect_database):

1. **Input Validation:** Validates request payload using Pydantic (Literal['hospital', 'ecommerce']).
2. **File Check:** Verifies physical SQLite file existence in data/; raises 404 if missing.
3. **Dynamic Inspection:** Binds sqlalchemy.inspect() to inspect tables and loops through columns to compile a clean dictionary mapping: {table: [{name: col, 'type': type}]}.
4. **Session Registration:** Generates a UUID session_id and records a SessionModel row in meta.db.
5. **Cache Population:** Invokes set_session() to store tables, schema, and path in RAM.
6. **Response:** Returns ConnectDBResponse with session_id, status='connected', and tables.

Design Decision: Schema discovery is completely dynamic. The code contains no hardcoded table or column names; it discovers schema directly from the database engine.

Honest Limitation: Currently restricted to local demo databases. It does not yet accept user-uploaded SQLite files or remote JDBC/SQLAlchemy connection strings (PostgreSQL/MySQL).

Judge Question: "If a database has 500 tables and 10,000 columns, won't extracting and storing the whole schema blow up your prompt?"

Model Answer: "Yes, full schema injection on massive databases exceeds LLM token limits and harms accuracy. Our planned Phase 5/6 improvement is Schema Pruning using Vector Search (RAG): we index table/column descriptions in FAISS and retrieve only the top 3-5 relevant table schemas per query."

5. app/services/sql_generator.py — Core LLM Translation & Prompt Engineering

What It Does: The translation core of the architecture. It retrieves cached schema context, constructs a grounded code-generation prompt, calls Google's gemini-3.6-flash model, sanitizes the raw output, and parses the structured JSON payload containing SQL, explanation, and confidence.

Internal Mechanics:

- **_clean_json_string(text):** Strips markdown code fences (```json ... ``` or ``` ... ```) and surrounding whitespace using regular expressions.
- **_format_schema_for_prompt(schema):** Converts structured schema dictionaries into human-readable text (e.g. Table 'patients': id (INTEGER), name (TEXT)...).
- **_validate_result(data):** Validates that output is a dictionary with non-empty sql and explanation strings, casting confidence to a float.
- **generate_sql(session_id, nl_question):** Compiles prompt, invokes gemini-3.6-flash, parses JSON, and contains an automated self-healing retry prompt if JSON parsing fails on attempt 1.

The Exact Prompt Sent to Gemini 3.6 Flash:

You are an expert SQLite SQL engineer.
 Given the following SQLite database schema:
 Table 'patients': id (INTEGER), name (VARCHAR), age (INTEGER), gender (VARCHAR), diagnosis (VARCHAR)
 Table 'doctors': id (INTEGER), name (VARCHAR), specialty (VARCHAR)

User Question: "{nl_question}"

Instructions:

1. Generate a valid, executable SQLite query that accurately answers the user's question.
 If the user asks to delete, insert, or update data, generate that query directly.
2. Return ONLY a single valid JSON object. Do not include markdown code fences, backticks, or extra text.
3. The JSON object must strictly have this exact structure:

```
{
  "sql": "SQL query here",
  "explanation": "Brief explanation of why this query answers the question",
  "confidence": 0.95
}
```

Why Structured / Code-Generation Prompting & JSON Mode?

- **Deterministic Code Generation:** Unlike open-ended conversational prompting, code generation treats the LLM as a compiler. We constrain the dialect (SQLite), ground the context (schema), and demand exact structure.
- **Why JSON over Freeform Text:** A backend pipeline cannot reliably parse conversational text ('Sure, here is your query: SELECT...'). Returning raw SQL alone discards the explanation and confidence metrics. Returning a structured JSON object cleanly packages the executable code (sql), user-facing reasoning (explanation), and risk assessment (confidence) in a machine-parseable contract.

The Phase 3 Bug Story (Great Presentation Narrative):

In our initial prompt, Instruction 1 strictly commanded: *'Generate a valid, executable SQLite SELECT query.'* When we tested Case B with a write query (*'Delete all patients older than 90'*), Gemini followed our prompt instruction rather than user intent and generated: `SELECT * FROM patients WHERE age > 90` to 'view' the patients to be deleted! The model actively masked the user's write intent, bypassing the test suite.

The Fix: We modified the instruction to generate write statements directly (`DELETE FROM ...`) when requested. This allowed our dedicated AST validator (`sql_validator.py`) to intercept the operation, block execution, and return a clean error without modifying the database. **Lesson for Judges:** Never rely on the LLM prompt as your security boundary. Prompts translate intent; deterministic code enforces security.

Judge Question: "Why didn't you use Gemini's native response_schema or response_mime_type='application/json'?"

Model Answer: "We chose a decoupled prompt-and-sanitization architecture with automated retry fallbacks to keep our pipeline model-agnostic. This design enables us to switch LLM providers (e.g. Claude 3.5, GPT-4o) or run a self-hosted open-source model (DeepSeek-Coder via Ollama) without refactoring SDK-specific schema bindings."

6. app/services/sql_validator.py — AST Security & Read-Only Enforcement

What It Does: Serves as the deterministic security gateway. It parses the generated SQL string into an Abstract Syntax Tree (AST) using `sqlglot`, verifying syntax correctness and strictly enforcing read-only (SELECT) queries while blocking any mutation, insertion, or administrative commands.

Validation Logic (`validate_sql`):

1. **Empty Check:** Rejects empty or whitespace-only inputs immediately as `syntax_error`.

2. **AST Parsing:** Executes `sqlglot.parse(sql_string, read='sqlite')`. Catches any `ParseError` and returns a clean syntax error dictionary.
3. **Token Clean-Up:** Strips trailing null statements (such as extra semicolons).
4. **Statement Type Verification:** Inspects each statement node: if not `isinstance(stmt, (exp.Select, exp.Query))`: return `{'valid': False, 'reason': 'write_not_supported', 'message': 'This version only supports read (SELECT) queries.'}`.
5. **Stacked Query Prevention:** Checks if `len(statements) != 1` to block multi-statement SQL injection attacks (e.g. `SELECT 1; DROP TABLE users;`).
6. **Success:** Returns `{'valid': True}`.

Design Decision (Why sqlglot over Regex?): Regex keyword matching is notoriously fragile. A benign query like `SELECT * FROM orders WHERE status = 'DROP_SHIP'` would trigger a false positive on 'DROP'. Conversely, attackers can bypass regex with comments (`/*comment*/DROP table`). `sqlglot` analyzes semantic syntax trees with mathematical certainty.

Honest Limitation: Lacks semantic schema checking. `SELECT fake_col FROM patients` is syntactically a valid `SELECT AST`, so it passes validation and relies on the execution engine to catch the missing column.

Judge Question: "Does your validator catch write operations disguised inside CTEs (Common Table Expressions)?"

Model Answer: "Yes. In `sqlglot`, statement classification inspects the root operational node. In a statement like `'WITH cte AS (SELECT 1) DELETE FROM patients...'`, the root node is an `exp.Delete` instance, which fails our `exp.Select/exp.Query` type check and is rejected."

7. `app/services/execution_engine.py` — Safe Execution & Error Containment

What It Does: Executes validated read-only queries against the session's target SQLite database and extracts tabular rows into JSON-serializable dictionaries. It guarantees crash resilience by catching database-level runtime exceptions and returning clean error dictionaries.

Execution Mechanics (`run_select`):

1. **Session & File Lookup:** Validates active session and confirms target database file exists on disk.
2. **Row Factory Configuration:** Opens connection with `conn.row_factory = sqlite3.Row` to enable dictionary-like column access.
3. **Execution & Extraction:** Runs `cursor.execute(sql)`, calls `cursor.fetchall()`, and uses list comprehension `[dict(row) for row in rows]` to serialize data.
4. **Exception Containment:** Wraps execution in `try...except Exception as exc:`, returning `{'error': str(exc)}` rather than crashing.
5. **Connection Cleanup:** Employs a `finally` block to ensure `conn.close()` is always executed, preventing lock leaks.

Design Decision: Uses lightweight raw `sqlite3.Row` instead of SQLAlchemy ORM models. Because dynamically generated SQL produces varying output column sets for every query, raw row dictionary mapping provides maximum performance and flexibility.

Honest Limitation: No query timeout or pagination guardrails. An accidental Cartesian join or full scan across a million-row table could cause high latency or memory spikes without an enforced `LIMIT`.

Judge Question: "What prevents a user query from running indefinitely or causing a Denial of Service on the database?"

Model Answer: "In this phase, we prioritized crash-proofing against errors and syntax failures. For production, we would add two safeguards: an execution timeout using `sqlite3.set_progress_handler` to abort queries after 3 seconds, and an AST transformation in `sqlglot` that injects an automatic `LIMIT 100` clause."

8. app/routers/query.py — Orchestration Pipeline & Contract Normalization

What It Does: The central orchestration router exposing POST `/api/query`. It coordinates session retrieval, Gemini SQL generation, AST validation, execution, and telemetry logging, returning a reliable, uniform JSON response contract for the frontend.

Step-by-Step Pipeline Flow (`handle_query`):

1. **Session Verification:** Validates `payload.session_id` in `session_store` (returns 404 if missing).
2. **Generation Step:** Calls `generate_sql()` to retrieve `sql`, `explanation`, and `confidence`.
3. **Validation Gate:** Calls `validate_sql(sql)`. If invalid (write query or syntax error): logs attempt to `meta.db` with `confidence=0.0` and returns a 200 OK `QueryResponse` with the validator's friendly explanation and `result=[]`.
4. **Execution Step:** If valid, calls `run_select(sql)`. If database execution throws an error: logs failure to `meta.db` and returns a 200 OK `QueryResponse` with the error explanation and `result=[]`.
5. **Success Path:** Commits successful query to `meta.db` and returns populated rows in `result`.

Design Decision: Consistent client-side contract. The endpoint avoids returning 500 server crashes or 400 errors for query failures. It returns a standardized 200 OK with `confidence: 0.0` and user-friendly explanations, allowing the frontend chat UI to gracefully display feedback without breaking.

Honest Limitation: No automated feedback retry loop yet. If a query fails due to a database schema mismatch (e.g. non-existent column), the error is surfaced to the user immediately rather than being fed back to Gemini for self-correction.

Judge Question: "Why return HTTP 200 with confidence: 0 instead of returning HTTP 400 or 422 on blocked queries?"

Model Answer: "In conversational AI products, a query blocked by policy or requesting unavailable data is a conversational event, not a network transport failure. Returning HTTP 200 with `confidence: 0`, the attempted SQL, and a friendly message allows the chat interface to render the agent's explanation naturally without triggering generic client error banners."

The 30-Second Jury Elevator Pitch (Memorize This!):

"We built an end-to-end, crash-proof Natural Language to SQL engine. It connects to any SQLite database, introspects its schema dynamically, and uses Gemini 3.6 Flash to translate user questions into SQL.

Unlike fragile demos that crash on bad inputs or risk destructive database writes, we implemented a decoupled security and execution pipeline: every query must pass through an Abstract Syntax Tree (AST) validator using `sqlglot` that strictly allows read-only `SELECT` queries. If a user asks to delete or modify data, the AST validator detects the write intent and blocks execution before it ever reaches the database.

Finally, our execution engine wraps all database calls in a crash-proof containment layer, ensuring the server stays up and the user receives a clear, friendly explanation every single time."